



WOLLO University

Department of software engineering

Blueprint based exit material for

Software testing, verification &

Quality assurance

Prepared by:

@Neyatibeb

KOMBOLCHA, Ethiopia

2015 E.C

❖ Analyse software requirement

- Software requirements analysis is a major SDLC approach and a leading step in the [Software development Life cycle](#).
- Software requirements consist of
 1. functional
 - describe the functionality of a system or system services. It depends on the type of software we are using and expected users. It explains an activity or process that the system must perform. looking at this from the business users' standpoint
 2. Non-functional requirements.
 - This defines the system properties and constraints. It describes the operational characteristics related to the system. 0
- Software requirements analysis is the process of discovering, understanding, documenting, and managing the requirements of a software system. It is a critical part of the software development process, as it ensures that the system meets the needs of its users and stakeholders.
- Its purpose is to identify the needs of the users and stakeholders, Document the requirements, Manage the requirements etc
- The technique to analyses SW requirement are Interviewing users and stakeholders, Observing users and stakeholders in their work environment, Creating use cases and Writing user stories.
- Here are some additional tips for software requirements analysis:
 - Start with the end in mind. What are the goals of the software system? What are the features that the users and stakeholders need?
 - Be clear and concise. The requirements should be easy to understand and interpret.
 - Be complete. The requirements should cover all of the features that the users and stakeholders need.
 - Be consistent. The requirements should be consistent with each other.
 - Be feasible. The requirements should be feasible to implement.
 - Be verifiable. The requirements should be verifiable, so that you can determine whether they have been met.
-

❖ Appropriate testing methodology

- The most common testing methodologies:
 - Waterfall methodology: This is a traditional testing methodology that follows a linear sequence of steps.
 - V-model methodology: This methodology combines waterfall methodology with verification and validation activities.
 - Spiral methodology: This methodology is an iterative approach to testing that allows for feedback and continuous improvement.
 - Agile methodology: This methodology is a lightweight approach to testing that is based on iterative development and continuous delivery.
- Choosing an appropriate testing methodology:

- Consider the size and complexity of the software. More complex software will require a more rigorous testing methodology.
- Consider the target audience. If the software is for a specific audience, you will need to tailor the testing methodology to their needs.
- Consider the budget. More rigorous testing methodologies can be more expensive.
- Consult with a testing expert. A testing expert can help you to choose the right testing methodology for your project.

❖ **Develop comprehensive test plan**

- Test planning, the most important activity to ensure that there is initially a list of tasks and milestones in a baseline plan to track the progress of the project. It also defines the size of the test effort. It is the main document often called as master test plan or a project test plan and usually developed during the early phase of the project.
- Test plan component
 - Test plan identifier
 - Introduction
 - Item to be tested
 - Feature to be tested
 -
- To develop comprehensive test plan we follow the step
 - Product analysis. This involves understanding the product requirements, features, and functionality. You need to identify the critical features that need to be tested thoroughly, as well as any potential risks or areas of concern.
 - Designing the test strategy. This involves determining the overall approach to testing, such as the types of tests that will be performed, the test levels, and the test environments. You also need to define the test criteria, which are the specific conditions that must be met in order for the test to pass.
 - Defining the objectives. What do you hope to achieve with the test plan? Do you want to verify that the product meets the requirements? Identify and fix defects? Or something else?
 - Establishing test criteria. What are the specific conditions that must be met in order for the test to pass? This includes things like expected results, input data, and environmental conditions.
 - Planning resource allocation. How much time and manpower will be needed to execute the test plan? You need to consider the number of tests that need to be performed, the complexity of the product, and the availability of resources.
 - Planning setup of test environment. This involves creating a test environment that is representative of the production environment. This includes setting up the hardware, software, and data that will be used for testing.
 - Determining test schedule and estimation. When will the tests be performed? How long will they take? You need to create a schedule that takes into account the availability of resources and the complexity of the product.
 - Establishing test deliverables. What documentation will be produced as a result of the testing? This includes things like test reports, defect logs, and test cases.

❖ Implement test case with appropriate tool

- Test Case is a simple pair of <input, expected outcome>
- Test case classify in to
 1. State less system – to compile a program it does not need to know about the program in compiled previously. The test case is simple. The outcome depends solely on current input. Example: compile
 2. State oriented - where the program outcome depends both on the current state of the system and the current input. Test cases are not that simple. The outcome depends both on the current state of the system and the current input. Example: ATM
- Test Set: A group of related tests is sometimes referred to as a test set
- Test Suite: Is a high-level concept, grouping together hundreds or thousands of tests related by what they are intended to test
- Test Log: Defines the result of test cases in terms of passed/failed after execution of the test case on the application build.
- Test Bed: is an environment that contains all the hardware and software needed to test a software component or a software system. This includes the entire testing environment
- Test Oracle: is a document, or piece of software that allows testers to determine whether a test has been passed or failed. A program or a document that produces or specifies the expected outcome of a test. It a mechanism that verifies the correctness of program outputs.
- Error: is a mistake, misconception, or misunderstanding on the part of a software developer.
- Faults: is introduced into the software as the result of an error. It is an anomaly in the software that may cause it to behave incorrectly, and not according to its specification. Faults or defects are sometimes called —bugs
- A failure: is the inability of a software system or component to perform its required functions within specified performance requirements. During execution of a software component or system, a tester, developer, or user observes that it does not produce the expected results.

❖ Compute test coverage

- Test coverage is defined as a statistic that indicates the quantity of testing completed by a collection of tests. It entails testing every requirement at least once and is clearly the responsibility of the QA team. Whereas Code Coverage refers to unit testing procedures that must target all parts of the code at least once and are performed by developers.
- To determine test coverage, follow the methods outlined below –
 - **Step 1** – The total number of lines of code in the program quality you are assessing.
 - **Step 2** – The total amounts of lines of code that all test cases are presently executing.

$$\text{Test Coverage (\%)} = (\text{Number of Lines Covered} / \text{Total Number of Lines}) * 100$$

Example:- If you have 500 lines of code in a system component and 50 lines run across all current test cases, your test coverage is –

$$10\% = (50 / 500) * 100$$

- To achieves test coverage by
 - Use of static review techniques such as peer reviews, inspections, and walkthroughs.

- By converting ad hoc flaws into executable test cases
- Test coverage may be done at the code or unit test level by using automated code coverage or unit test coverage technologies.
- Functional test coverage may be accomplished with the assistance of appropriate test management technologies.
-
- The higher your test coverage, the more confident you can be that your code is free of defects. However, it is important to note that test coverage is not a perfect measure of quality. Even if your test coverage is high, there is still a chance that your code may contain defects
- There are a number of factors that can affect test coverage, including:
 - The size of your project
 - The complexity of your code
 - The type of testing you are performing
 - The quality of your test suite

❖ Statistical technique to evaluate defect and fault

- There are a number of statistical techniques that can be used to evaluate defects and faults in software. Some of the most common methods include:
 - Linear regression can be used to identify the relationship between defect density and various software metrics, such as the number of lines of code, the number of modules, and the complexity of the code.
 - Logistic regression can be used to predict the likelihood of a defect occurring, based on a set of software metrics.
 - Neural networks can be used to learn the relationship between defect density and software metrics, and then use this knowledge to predict the likelihood of a defect occurring.
 - Fault tree analysis (FTA) is a top-down approach to identifying the potential causes of a failure. FTA uses a graphical representation to show how different events can combine to cause a failure.
 - Failure mode and effect analysis (FMEA) is a bottom-up approach to identifying the potential causes of a failure. FMEA uses a table to list the potential failure modes, their effects, and their likelihood of occurrence.
- The choice of statistical technique will depend on the specific application. For example, linear regression may be a good choice if you want to identify the relationship between defect density and a few key software metrics. Logistic regression may be a better choice if you want to predict the likelihood of a defect occurring, based on a larger set of software metrics.
- Mutation testing is a technique for testing the effectiveness of a software testing suite. Mutation testing injects artificial defects into the software, and then runs the testing suite to see if the defects are detected.
- Code coverage analysis is a technique for measuring the extent to which a software testing suite has exercised the code. Code coverage analysis can be used to identify areas of the code that are not being adequately tested.

- Static analysis is a technique for analysing the source code of a software program without actually running the program. Static analysis can be used to identify potential defects in the code, such as potential errors, potential security vulnerabilities, and potential performance problems.
- The choice of method will depend on the specific application. For example, mutation testing may be a good choice if you want to ensure that your testing suite is effective at detecting defects. Code coverage analysis may be a good choice if you want to identify areas of the code that are not being adequately tested. Static analysis may be a good choice if you want to identify potential defects in the code before the program is actually run.